Inteligență Artificială

Introducere în rețele neurale

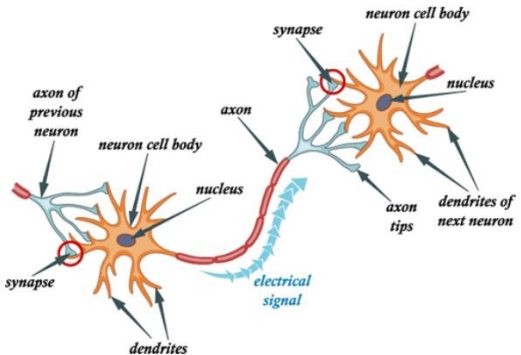
Slides: Mihai Trăscău, Mihai Nan

- Scurtă istorie
- Neuron
- ANN
- Perceptron
- MLP
- Graf
- Backpropagation
- Regularizare

- **1943** → McCulloch, Pitts - model simplu (binary devices with fixed thresholds)
- **1960** → Rosenblatt propune Perceptron (feed-forward ANN) - enunță și demonstrează teorema de convergență a perceptronului
- **1960** → Widrow, Hoff - ADALINE (ADAptive LInear Element)
- **1969** → Minsky, Papert demonstrează limitările perceptronului
- **1970** → Abandon masiv al rețelelor neurale (în laboratoarele de cercetare)
- **1980** → Sunt reluate cercetările privind rețelele neurale
- **1980** → Fukushima propune Neocognitron
- **1986** → Hopfield propune funcția de energie. A pus în evidență noțiunea de memorie ca o mulțime de atractori dinamici stabili (rețele recurente)
- **1986** → Rumelhart propune perceptroni multinivel și backpropagation pentru antrenarea rețelelor (rețele feed-forward)

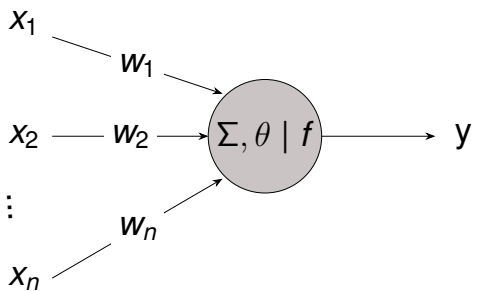
- **1988** → Grossberg și Carpenter propun algoritmi de rezonanță, rețele ART (Adaptive Resonance Theory)
- **1988** → Anderson și Kohonen (independent) propun modele cu tehnici asociative
- **1989** → Yann LeCun aplică backpropagation rețelelor adânci (deep learning)
- **1998** → Apar îmbunătățiri ale arhitecturilor (e.g. LeNet-5 cu straturi convoluționale)
- **2009** → Ranja, Madhavan și Andrew Ng propun utilizarea GPU (Graphics Processing Unit) pentru antrenarea rețelelor neurale
- **2009** → Dan Cireșan și Jürgen Schmidhuber au obținut rezultate foarte bune pe setul de date MNIST folosind o rețea convoluțională mare
- **2012** → Alex Krizhevsky, Ilya Sutskever and Geoffrehy Hinton au folosit o rețea convoluțională și mai mare pentru a obține rezultate remarcabile (relativ la celelate ale vremii) pe setul de date ImageNet
- **... - prezent** → Explozia continuă până astăzi, atât ca arhitecturi și tehnici, dar și prin scalare

- Creierul uman are, în medie, 10^{10} neuroni
- Numărul de conexiuni sinaptice (de intrare) ale unui neuron este între 10^3 și 10^4
- Semnalele de intrare sunt recepționate prin dendrite și au un efect cumulativ
- Dacă se depășește un prag de potențial, un semnal de ieșire este trecut prin axon



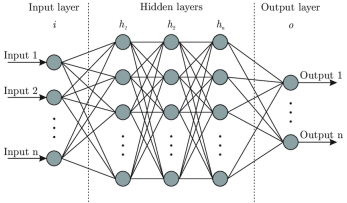
- Modelul neuronului artificial al lui McCulloch și Pitts (1943)
- Neuronul prezice ieșirea $y = f(\sum_{i=1}^{n} w_i x_i - \theta)$
- w_i reprezintă intensitatea legăturii (conexiunii) de la intrarea x_i
- θ valoarea de prag peste care neuronul se activează
- f este funcția de decizie. Când $f > 0$ neuronul este excitat, dacă $f < 0$ neuronul este inhibat iar când $f = 0$ nu există conexiune între neuroni

$$f(x) = \begin{cases} 1, & \text{dacă } x \geq 1 \\ 0 \text{ (sau -1)}, & \text{altfel} \end{cases}$$

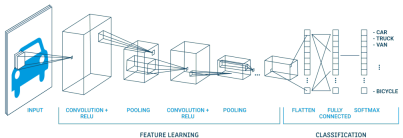


- Tipul de învățare: supervizat, semi-supervizat (auto-supervizat), nesupervizat
- Topologia (arhitectura) rețelei
- Numărul de straturi (niveluri)
- Tipuri de rețele (după modul de interconectare a activărilor)
  - Feed-forward Networks - conexiuni doar "înainte"
  - Recurrent Networks - conexiuni ciclice
  - Graph Neural Networks - conexiuni pe baza unui graf
  - Transformer Networks - mecanisme de atenție
- Tipul intrărilor și ieșirilor
- Algoritmi de optimizare folosiți în antrenare (pentru găsirea ponderilor)
- Funcția de activare
- Funcția de eroare

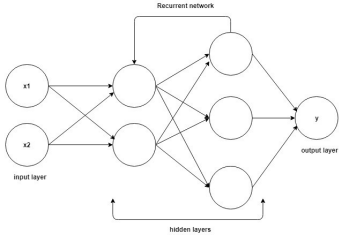
- Multi-Layer Perceptron (MLP)



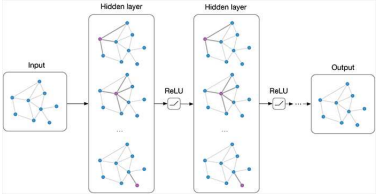
- Convolutional Neural Networks (CNN)



- Recurrent Neural Networks (RNN)



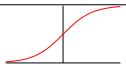
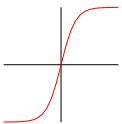
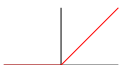
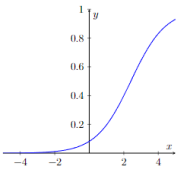
- Graph Neural Networks (GNN)



- Hiperparametri model:
  - Număr nivele ascunse
  - Numărul de neuroni pe fiecare nivel ascuns
  - Funcția de activare pe fiecare nivel
  - ...
- Hiperparametri antrenare:
  - Inițializarea ponderilor
  - Optimizator (e.g. viteza de învățare, regim viteză de învățare, momentum)
  - Numărul de epoci / iterații
  - Dimensiune batch
  - ...

- Ipoteza de învățare supervizată:
  - Se dă setul de date cu n exemple de învățare: $\{x_i, y_i\}_{i=1}^{n}$
  - Se alege o funcție de decizie $\hat{y} = f_\theta(x_i)$
  - Se alege o funcție de eroare $J(\hat{y}_i, y_i) \in \mathbb{R}$
  - Să se determine parametrii modelului astfel încât să se minimizeze valoarea funcției de eroare:

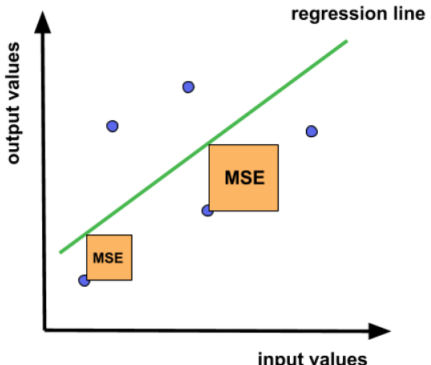
$$\theta^* = \underset{\theta}{argmin} \sum_{i=1}^{n} J(f_\theta(x_i), y_i)$$

Numele	Funcția	Derivata	Graficul
Sigmoid	$\sigma(x) = \frac{1}{1+e^{-x}}$	$f'(x) = f(x)(1 - f(x))$	
tanh	$\sigma(x) = \frac{e^x - e^{-x}}{e^z + e^{-z}}$	$f'(x) = 1 - f(x)^2$	
ReLU	$f(x) = \begin{cases} 0 & \text{if } x < 0 \\ x & \text{if } x \geq 0. \end{cases}$	$f'(x) = \begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0. \end{cases}$	
Softmax	$f(x) = \frac{e^x}{\sum_i e^x}$	$f'(x) = \frac{e^x}{\sum_i e^x} - \frac{(e^x)^2}{(\sum_i e^x)^2}$	

- Eroarea pătratică medie

$$\mathcal{L}(\hat{y}, y_i) = \frac{1}{N}\sum_{1}^{N}(h(x^{(i)}) - y^{(i)})^2 \in \mathbb{R}$$

cu h funcția de decizie și N numărul de exemple



- Cross Entropy – măsoara diferența între 2 distribuții de probabilitate p și q

$$CE(p, q) = -\sum_{x} (p(x) \log q(x))$$

- Pentru probleme de clasificare cu C clase – Cross entropy loss

$$CE(y, f) = -\sum_{j=1}^{C} y_j \log(f(x_j, w))$$

- Pentru probleme de clasificare in care $C = 2$

$$CE(y, f) = -y \log(f(x, w)) - (1 - y) \log(1 - f(x, w))$$

- Funcția de eroare se calculează prin media erorilor din toate cele N exemple de învățare:

$$\mathcal{L}(\hat{y}, y^{(i)}) = -\frac{1}{N} \sum_{i=1}^{N} \left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$$

- Multinomial Logistic Loss/Regression
- Este o funcție de clasificare Softmax combinate cu Cross entropy loss
- Ieșirea rețelei este interpretată ca o distribuție de probabilitate peste cele C (k) clase
- Pentru a transforma scorurile într-o distribuție de probabilitate, se aplică exponent și se normalizează.
- Softmax Classifier

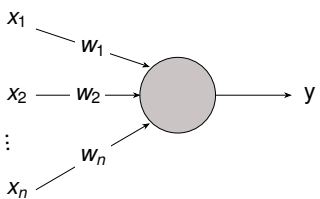
$$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

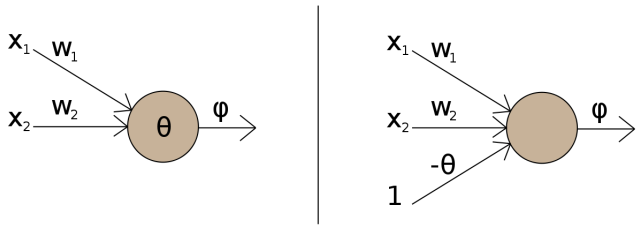
unde $s_i = f(x_i, w)$

- În cazul unei clasificări multi-clasa se rețin numai clasele. Exista un singur element care este diferit de 0 deci se elimina celelalte elemente din suma ($y_j = 0$).

$$\mathcal{L}_i(f(x_i, w), y_i) = -\log P(Y = y_i|X = x_i) = -\log\left(\frac{e^{s_i}}{\sum_j e^{s_j}}\right)$$

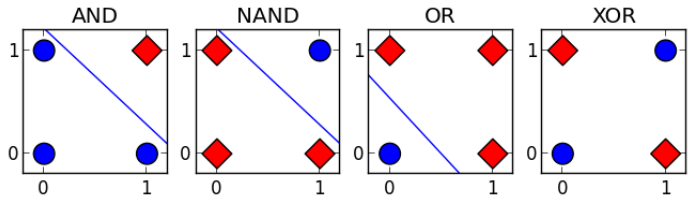
- Modelul a fost propus inițial în 1962 de către Rosenblatt.
- Modelul a fost actualizat de Minsky și Papert.
- Modelul are N intrări binare (fiecare exemplu are dimensiune N) și avem M exemple de intrări $X^{(i)} = [x_1^{(i)}, x_2^{(i)} \ldots, x_N^{(i)}], \forall i \in \{1, 2, \ldots, M\}$.
- Ieșiri $\hat{y} = +1/0$ (sau $+1/-1$).
- Modelul utilizează ca funcție de activare funcția treaptă (limitator).





- Unitate computațională cu valoare de prag θ care pentru intrări $x_1, \ldots, x_n$ și ponderi $w_1, \ldots, w_n$ produce la ieșire $+1$ dacă $\sum_{k=1}^{N} w_k x_k \geq \theta$ și 0 (sau -1) în caz contrar.
- Separă spațiul intrărilor în 2 zone: pct pt care rezultatul este 1 și pct pt care rezultatul este 0.

Caracteristicile Modelului Perceptron



- Funcțiile calculabile de perceptron sunt acele funcții pentru care punctele cu valoarea funcției 1 (◇) pot fi separate de punctele cu valoarea funcției 0 (○) – liniar separabile
- Minsky și Papert [1969] au pus în evidență limitările modelului computațional al perceptronilor

Intrări: $Dataset = \{X^{(i)}, y^{(i)}\}_{i=1}^{N}$, $maxEpochs$, η

Ieșire: Valorile pentru $w_1, w_2, \ldots w_D$ și b

1. $w_d \leftarrow 0, \forall d = 1 \ldots D; b \leftarrow 0$
2. **pentru** $epoch \leftarrow 1 \ldots maxEpochs$
3. **pentru fiecare** $(x, y) \in Dataset$
4. $\hat{y} \leftarrow f\left(\sum_{d=1}^{D} w_d \cdot x_d + b\right)$
5. **dacă** $\hat{y} \neq y$
6. $w_d \leftarrow w_d + \eta \cdot \Delta w, \forall d \in 1 \ldots D$
7. $b \leftarrow b + \eta \cdot \Delta w$

- Plecând de la funcția de ieșire:

$$y^{(i)} = f\left(\mathbf{xw} + b\right) = f\left(x_1^{(i)} \cdot w_1 + x_2^{(i)} \cdot w_2 + \cdots + x_d^{(i)} \cdot w_d + b\right)$$

- Rescriem în formă matriceală

$$y^{(i)} = f\left(\begin{bmatrix} 1 & x_1^{(i)} & x_2^{(i)} & \dots & x_d^{(i)} \end{bmatrix} \begin{bmatrix} b \\ w_1 \\ w_2 \\ \vdots \\ w_d \end{bmatrix}\right)$$

- $w_j = w_j + \eta(y^{(i)} - \hat{y}^{(i)}) \cdot x_j^{(i)}$
- Dacă $\hat{y} = y \Rightarrow \Delta\mathbf{w} = 0$
- Dacă $\hat{y} = -y \Rightarrow \Delta\mathbf{w} = \pm 2\eta\mathbf{x}^{(i)}$

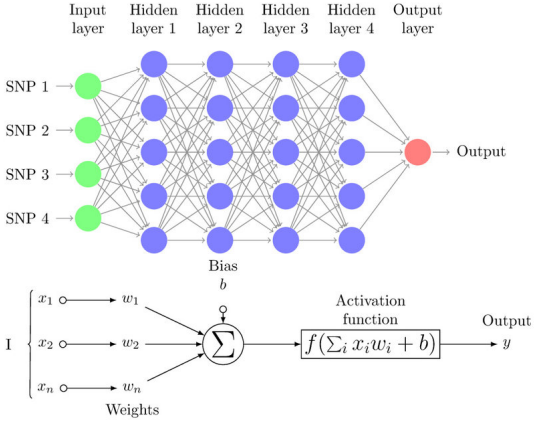
- Există două probleme mari ale modelului Perceptron:
  1. Funcție de activare nediferențiabilă
  2. Capacitate redusă a modelului
- Funcția de activare folosită nu este potrivită (poate fi prea *dură*).
- Pentru a atenua această problemă, Bernard Widrow și Tedd Hoff au propus așa-numitul Adaptive Linear Neuron (sau pur și simplu Adaline).
- Adaline folosește identitatea pentru funcția de activare:
  $\hat{y} = f\left(\mathbf{w}^T\mathbf{x}\right) = \mathbf{w}^T\mathbf{x}$

$$\frac{\partial J}{\partial w} = \frac{1}{2}\frac{\partial (y-\hat{y})^2}{\partial w} = \frac{1}{2}\sum_i 2(y_i-\hat{y}_i)\frac{\partial (y_i-\hat{y}_i)}{\partial w} =$$

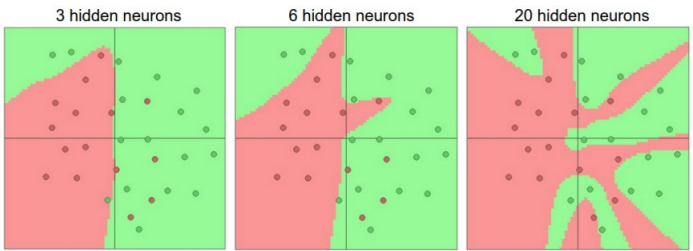
$$= \sum_i (y_i-\hat{y}_i)\frac{\partial}{\partial w_j}\left(y_i - \sum_j w_j x_j\right) = \sum_i (y_i-\hat{y}_i)(-x_i) =$$

$$= -\sum_i (y_i-\hat{y}_i)x_i$$

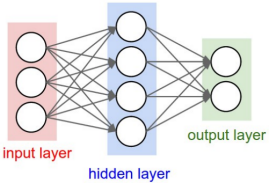
- Perceptronii multi-nivel (MLP) sau rețele backpropagation
- Sunt rețele neurale cu activare directă care conțin unul sau mai multe straturi de noduri între nodurile de intrare și nodurile de ieșire.
- Aceste straturi suplimentare reprezintă nivelele ascunse ale perceptronilor multi-nivel.



- Rețeaua neurală cu cel puțin un strat ascuns este un **aproximator universal** (poate aproxima orice funcție continuă pe $\mathbb{R}$, dându-se un număr suficient, dar finit, de neuroni, și o funcție de activare potrivită).
- Capacitatea rețelei crește cu adăugarea mai multor unități ascunse și mai multe straturi ascunse.



- Algoritmul de învățare al rețelelor backpropagation este format din doi pași principali:
  1. **Algoritmul Forward** – utilizat pentru inferență
     - O parcurgere directă a rețelei, de la intrări la ieșiri, în care se activează rețeaua și se determină valorile ieșirilor
  2. **Algoritmul Backward** – utilizat pentru antrenarea modelului
     - O parcurgere înapoi a rețelei, de la ieșiri la intrări, în care ieșirile calculate se compară cu ieșirile din exemple și se determină o estimare a erorii; această estimare a erorii este propagată înapoi și utilizată la actualizarea ponderilor.



- Output-ul rețelei neurale poate fi calculat ca:

$$h_j(x) = f\left(v_{j0} + \sum_{i=1}^{D} x_i v_{ji}\right) \text{ și } o_k(x) = g\left(w_{k0} + \sum_{j=1}^{J} h_j(\mathbf{x}) w_{kj}\right)$$

(j – indexul unității ascunse, k – indexul unității de ieșire, D - numărul de intrări, f, g – funcții de activare)

$$\sigma(z) = \frac{1}{1 + exp(-z)}, tanh(z) = \frac{exp(z) - exp(-z)}{exp(z) + exp(-z)}, ReLU(z) = max(0, z)$$

- Ne dorim să determinăm ponderile rețelei astfel încât:

$$\mathbf{w}^* = \arg\min_{w} \sum_{n=1}^{N} loss(\mathbf{o}^n, \mathbf{t}^n)$$

unde $\mathbf{o} = f(\mathbf{x}; \mathbf{w})$ este predicția rețelei neurale.

- Trebuie să ne definim o funcție de eroare:
  1. **Squared loss**: $\sum_k \frac{1}{2} \left(o_k^{(n)} - t_k^{(n)}\right)^2$ (*regresie*)
  2. **Cross-entropy loss**: $-\sum_k t_k^{(n)} \log o_k^{(n)}$ (*clasificare*)
- Algoritmul de optimizare – **Gradient descent**

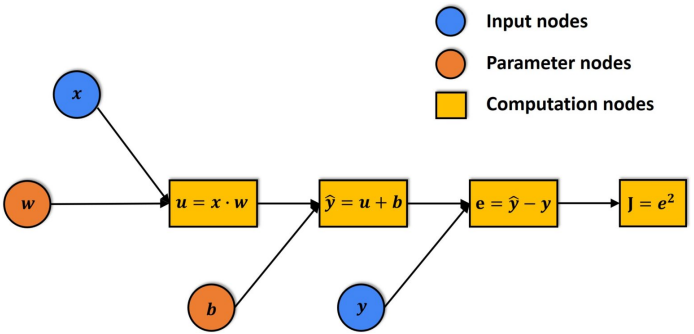
$$\mathbf{w}^{t+1} = \mathbf{w}^t - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{w}^t}$$

unde η este rata de învățare, iar $\mathcal{L}$ este funcția de eroare.

- Repetă până la convergență:
  - Pentru fiecare exemplu $(x^{(n)}, y^{(n)})$
    1. Aplicarea algoritmului Forward pentru calcularea predicției $(\mathbf{x}^{(n)} \rightarrow \mathbf{h}(\mathbf{x}^{(n)}) \rightarrow \mathbf{o}(\mathbf{h}(\mathbf{x}^{(n)})))$
    2. Calculul funcției de eroare $\mathcal{L}(\hat{y}^{(n)}, y^{(n)})$
    3. Calcularea gradientului funcției de eroare în raport cu ponderile modelului $\frac{\partial \mathcal{L}(\hat{y}^{(n)}, y^{(n)})}{\partial \mathbf{w}}$
    4. Actualizarea ponderilor $\mathbf{w}$ (în raport cu gradientul)

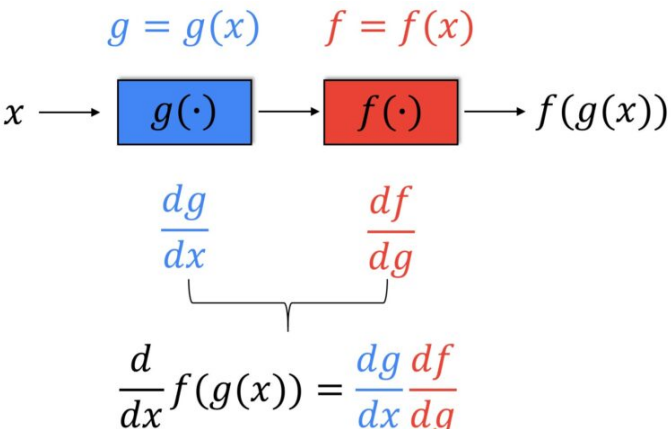
Modelul liniar

$$\hat{y} = x \cdot w + b; \mathcal{L} = (y - \hat{y})^2 = (y - x \cdot w - b)^2$$



- **noduri de intrare** – primesc valori din setul de date;
- **noduri de parametri** – rețin parametri modelului;
- **noduri de calcul** – utilizate pentru calcularea ieșirii (funcția de cost este stocată în nodul final)

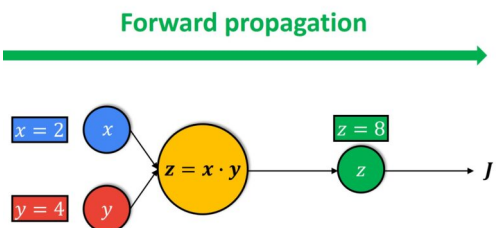
Chain rule


$g = g(x)$
$f = f(x)$
x
$g(\cdot)$
$f(\cdot)$
$f(g(x))$
$\frac{dg}{dx}$
$\frac{df}{dg}$
$\frac{d}{dx} f(g(x)) = \frac{dg}{dx}\frac{df}{dg}$

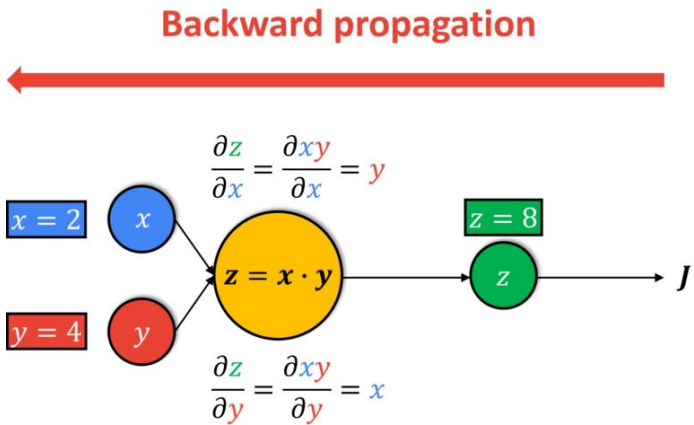
- Pentru a înțelege mai bine cum funcționează procesul de antrenare pentru o astfel de rețea și care este utilitatea grafului computațional, pornim de la un exemplu foarte simplu.

$$z = x \cdot y$$

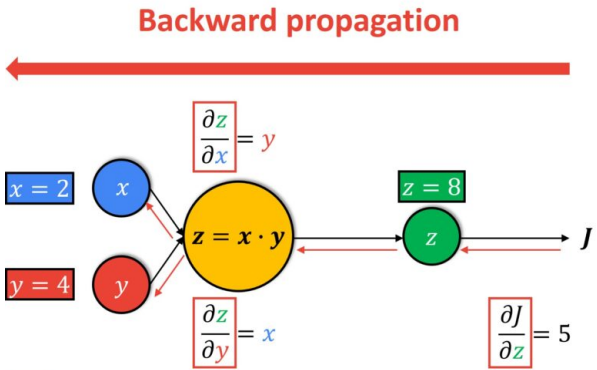
- Considerăm că avem un exemplu pentru care $x = 2$ și $y = 4$. Putem folosi următorul graf computațional pentru a calcula ieșirea z, iar pentru această ieșire calculăm eroarea (aplicând funcția de eroare).



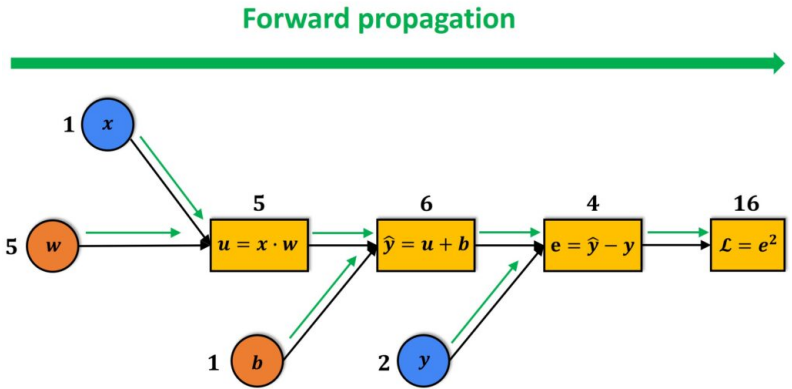
- După ce am obținut eroarea, putem să calculăm gradienții.



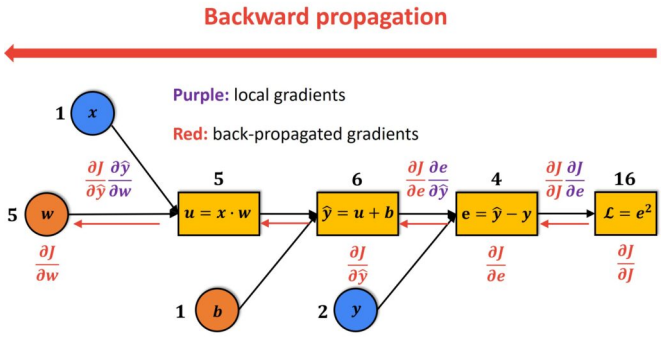
- Vom presupune că $\frac{\partial J}{\partial z} = 5$.



Modelul liniar



Modelul liniar



$$\frac{\partial J}{\partial e} = \frac{\partial J}{\partial J}\frac{\partial J}{\partial e} = \frac{\partial J}{\partial J}\frac{\partial e^2}{\partial e} = 1 \cdot 2e = 8$$

$$\frac{\partial J}{\partial \hat{y}} = \frac{\partial J}{\partial e}\frac{\partial e}{\partial \hat{y}} = \frac{\partial J}{\partial e}\frac{\partial (\hat{y} - y)}{\partial \hat{y}} 2e \cdot 1 = 8$$

$$\frac{\partial J}{\partial w} = \frac{\partial J}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial w} = \frac{\partial J}{\partial \hat{y}}\frac{\partial xw}{\partial w} = 2e \cdot x = 8$$

Gradient descent (revizitat)

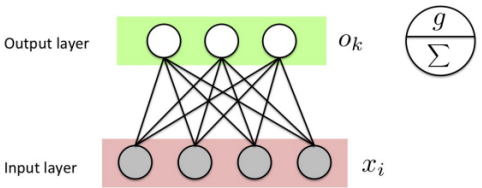
1. Pornim de la niște valori aleatoare pentru parametri ($w = 5, b = 1$).
2. Repetă până la convergență
   - 2.1 Calculează $\frac{\partial J}{\partial w}$
   - 2.2 Actualizează w

$$w \leftarrow w - \alpha \cdot \frac{\partial J}{\partial w}$$

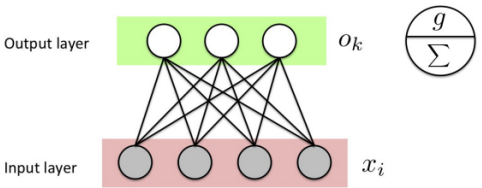
   - 2.3 Calculează $\frac{\partial J}{\partial b}$
   - 2.4 Actualizează b

$$b \leftarrow b - \alpha \cdot \frac{\partial J}{\partial b}$$

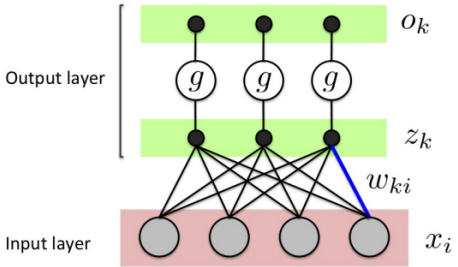
- Pornim de la o rețea neurală cu un singur strat.



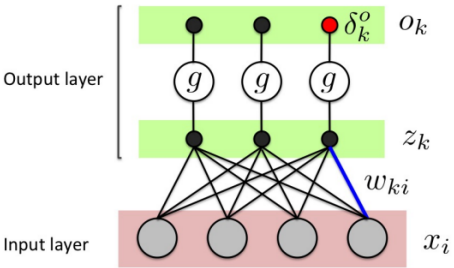
- Pornim de la o rețea neurală cu un singur strat.



- Pornim de la o rețea neurală cu un singur strat.

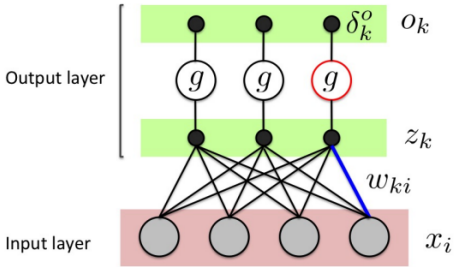


$$\frac{\partial \mathcal{L}}{\partial w_{ki}} = \frac{\partial \mathcal{L}}{\partial o_k} \cdot \frac{\partial o_k}{\partial z_k} \cdot \frac{\partial z_k}{\partial w_{ki}}$$



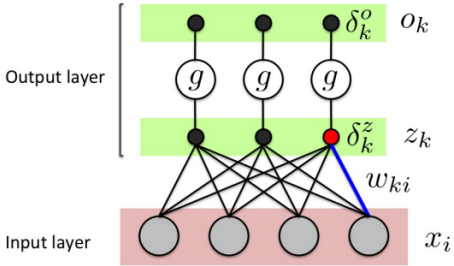
$$\delta_k^o = \frac{\partial \mathcal{L}}{\partial o_k}$$

$$\frac{\partial \mathcal{L}}{\partial w_{ki}} = \overbrace{\frac{\partial \mathcal{L}}{\partial o_k}}^{\delta_k^o} \cdot \frac{\partial o_k}{\partial z_k} \cdot \frac{\partial z_k}{\partial w_{ki}}$$



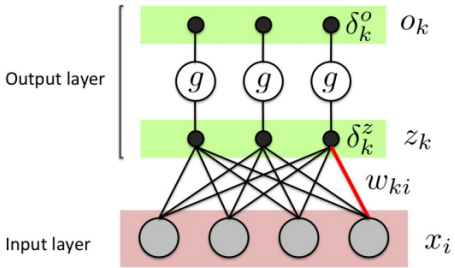
$\frac{\partial o_k}{\partial z_k}$

$$\frac{\partial \mathcal{L}}{\partial w_{ki}} = \frac{\partial \mathcal{L}}{\partial o_k} \cdot \frac{\partial o_k}{\partial z_k} \cdot \frac{\partial z_k}{\partial w_{ki}} = \delta_k^o \cdot \frac{\partial o_k}{\partial z_k} \cdot \frac{\partial z_k}{\partial w_{ki}}$$



$$\delta_k^z = \delta_k^o \cdot \frac{\partial o_k}{\partial z_k}$$

$$\frac{\partial \mathcal{L}}{\partial w_{ki}} = \frac{\partial \mathcal{L}}{\partial o_k} \cdot \frac{\partial o_k}{\partial z_k} \cdot \frac{\partial z_k}{\partial w_{ki}} = \overbrace{\delta_k^o \cdot \frac{\partial o_k}{\partial z_k}}^{\delta_k^z} \cdot \frac{\partial z_k}{\partial w_{ki}}$$



$$\frac{\partial z_k}{\partial w_{ki}} = \frac{\partial\left(w_{k0}+\sum_{j=1}^{J} x_j w_{kj}\right)}{\partial w_{ki}} = x_i$$

$$\frac{\partial \mathcal{L}}{\partial w_{ki}} = \frac{\partial \mathcal{L}}{\partial o_k} \cdot \frac{\partial o_k}{\partial z_k} \cdot \frac{\partial z_k}{\partial w_{ki}} = \delta_k^o \cdot \frac{\partial o_k}{\partial z_k} \cdot \underbrace{\frac{\partial z_k}{\partial w_{ki}}}_{x_i} = \delta_k^z \cdot x_i$$

- Considerăm funcția de eroare MSE care pentru un singur exemplu va avea formula: $\mathcal{L}\left(\hat{y}^{(n)}, y^{(n)}\right) = \frac{1}{2} \cdot \left(\hat{y}^{(n)} - y^{(n)}\right)^2$
- În cazul nostru, $\hat{y}^{(n)} = o_k^{(n)}$ pentru care $o_k^{(n)} = g(z_k^{(n)})$.
- Din primele 2 relații $\Rightarrow \frac{\partial \mathcal{L}}{\partial o_k^{(n)}} = o_k^{(n)} - t_k^{(n)}$
- Vom considera ca funcție de activare funcția sigmoid:

$$g\left(z_k^{(n)}\right) = \frac{1}{1 + e^{-z_k^{(n)}}} = \left(1 + e^{-z_k^{(n)}}\right)^{-1}$$

- Reamintim că pentru funcția sigmoid cunoaștem:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \text{ și } \sigma'(x) = \sigma(x)\left(1 - \sigma(x)\right)$$

- Astfel, vom avea:

$$\frac{\partial o_k^{(n)}}{\partial z_k^{(n)}} = o_k^{(n)}(1 - o_k^{(n)})$$

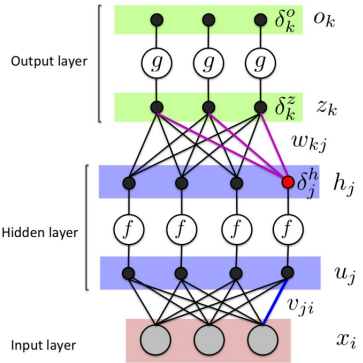
- Astfel, putem determina valoarea pentru gradient, utilizând relațiile obținute anterior:

$$\frac{\partial \mathcal{L}}{\partial w_{ki}} = \sum_{n=1}^{N} \frac{\partial \mathcal{L}}{\partial o_k^{(n)}} \cdot \frac{\partial o_k^{(n)}}{\partial z_k^{(n)}} \cdot \frac{\partial z_k^{(n)}}{\partial w_{ki}} = \sum_{n=1}^{N} \left(o_k^{(n)} - t_k^{(n)} \right) o_k^{(n)} \left(1 - o_k^{(n)} \right) x_i^{(n)}$$

- Aceasta valoare a gradientului ne spune cum trebuie să actualizăm ponderile rețelei neurale:

$$w_{ki} \leftarrow w_{ki} - \eta \cdot \frac{\partial \mathcal{L}}{\partial w_{ki}} = w_{ki} - \eta \cdot \sum_{n=1}^{N} \left(o_k^{(n)} - t_k^{(n)} \right) o_k^{(n)} \left(1 - o_k^{(n)} \right) x_i^{(n)}$$

- **Calculul gradientului pentru o rețea cu mai multe straturi**



$$\frac{\partial \mathcal{L}}{\partial w_{kj}} = \sum_{n=1}^{N} \frac{\partial \mathcal{L}}{\partial o_k^{(n)}} \cdot \frac{\partial o_k^{(n)}}{\partial z_k^{(n)}} \cdot \frac{\partial z_k^{(n)}}{\partial w_{kj}} = \sum_{n=1}^{N} \delta_k^{z,(n)} h_j^{(n)}$$

$$\frac{\partial \mathcal{L}}{\partial h_j^{(n)}} = \sum_{k} \frac{\partial \mathcal{L}}{\partial o_k^{(n)}} \cdot \frac{\partial o_k^{(n)}}{\partial z_k^{(n)}} \cdot \frac{\partial z_k^{(n)}}{\partial h_j^{(n)}} = \sum_{k} \delta_k^{z,(n)} w_{kj} = \delta_j^{h,(n)}$$

$$\frac{\partial \mathcal{L}}{\partial v_{ji}} = \sum_{n=1}^{N} \frac{\mathcal{L}}{\partial h_j^{(n)}} \cdot \frac{\partial h_j^{(n)}}{\partial u_j^{(n)}} \cdot \frac{\partial u_j^{(n)}}{\partial v_{ji}} = \sum_{n=1}^{N} \delta_j^{h,(n)} f'\left(u_j^{(n)}\right) \frac{\partial u_j^{(n)}}{\partial v_{ji}} = \sum_{n=1}^{N} \delta_j^{u,(n)} x_i^{(n)}$$

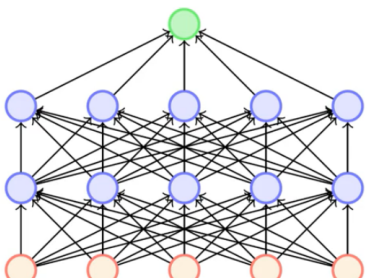
- Cum alegem funcțiile de activare și eroare?
- **Regresie**
  - Când utilizăm o rețea neurală pentru regresie, preferăm funcția sigmoid pentru activări și MSE ca funcție de eroare.
- **Clasificare binară**
  - Pentru clasificarea binară putem folosi funcția de eroare Cross-Entropy:

$$\mathcal{L} = -\sum_{n=1}^{N} y^{(n)} \log \hat{y}^{(n)} + \left(1 - y^{(n)}\right) \log \left(1 - \hat{y}^{(n)}\right) \text{ și } \hat{y}^{(n)} = \frac{1}{1 + e^{-z^{(n)}}}$$

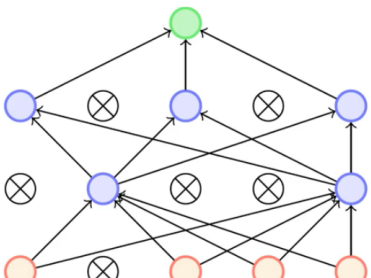
$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = \frac{(\hat{y} - y)}{\hat{y}(1 - \hat{y})}$$

$$\frac{\partial \hat{y}}{\partial z} = \hat{y}(1 - \hat{y}) \text{ și } \frac{\partial \mathcal{L}}{\partial z} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{z} = (\hat{y} - y)$$

Dropout

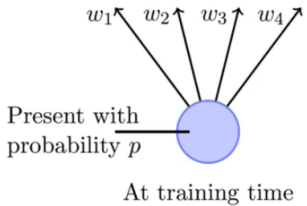


Original network

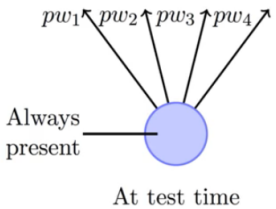


Network with some nodes dropped out

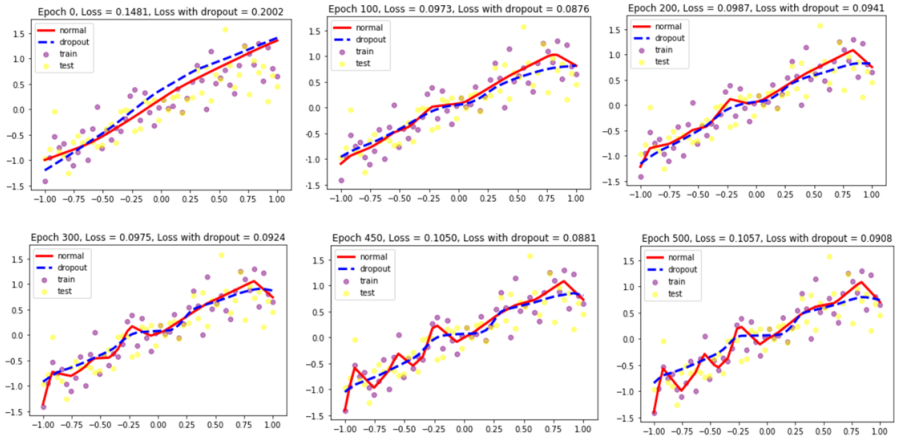
Dropout – *antrenare*



Dropout – *testare*



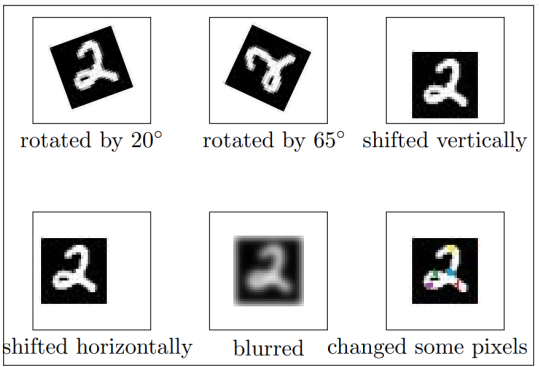
Dropout – Exemplu



Regularizare : Augmentare



label = 2



label = 2

Regularizare L_2

$$\tilde{\mathcal{L}}(\theta) = \mathcal{L}(\theta) + \frac{\alpha}{2}\|\theta\|_2^2$$

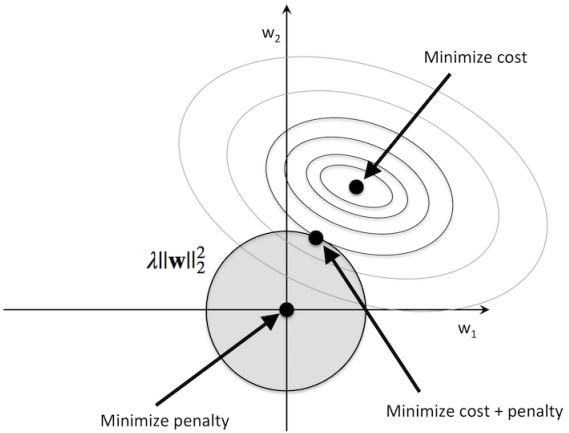
Pentru Gradient Descent vom avea:

$$\nabla\tilde{\mathcal{L}}(\theta) = \nabla\mathcal{L}(\theta) + \alpha \cdot \|\theta\|_2$$

Astfel, atunci când actualizăm parametri vom folosi:

$$\theta^{t+1} \leftarrow \theta^t - \eta\nabla\mathcal{L}(\theta^t) - \eta \cdot \alpha \cdot \theta^t$$

Interpretare geometrică pentru regularizarea L_2



Regularizare L_1

$$\tilde{\mathcal{L}}(\theta) = \mathcal{L}(\theta) + \alpha \cdot \|\theta\|_1$$

Pentru Gradient Descent vom avea:

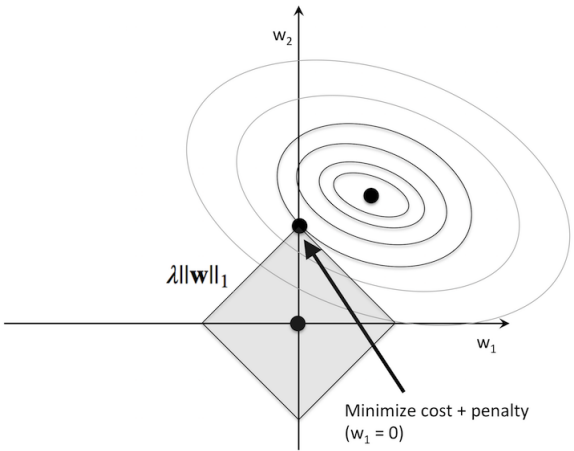
$$\nabla\tilde{\mathcal{L}}(\theta) = \nabla\mathcal{L}(\theta) + \alpha \cdot \frac{\partial\|\theta\|_1}{\partial\theta}$$

Astfel, atunci când actualizăm parametri vom folosi:

$$\theta^{t+1} \leftarrow \theta^t - \eta\nabla\mathcal{L}(\theta^t) + \eta \cdot \alpha, \text{ dacă } \theta^t > 0$$

$$\theta^{t+1} \leftarrow \theta^t - \eta\nabla\mathcal{L}(\theta^t) - \eta \cdot \alpha, \text{ dacă } \theta < 0$$

Interpretare geometrică pentru regularizarea L_1



Mulțumesc!



https://forms.gle/DJUdkejstkvyNRF5A

Feedbackul este binevenit!